Table 5: Comparison of Candidate LOTUS Join Plans and Gold Algorithm on the BioDEX Dataset Using Llama-70b

Method	RP@5	RP@10	Execution Time (s)	# LM Calls
LOTUS Join Plan 1	0.1541	0.170	12,563	27,687
LOTUS Join Plan 2 (chosen)	0.212	0.213	2,116	5,290
Gold Algorithm	N/A	N/A	2,144,560	6,092,500

Next, The AI UDF baseline, using row-wise LLM calls, is prohibitively expensive, and reflects the cost of our gold algorithm for joins. Its quadratic scaling of LLM call complexity with respect to dataset size leads to $1,000\times$ higher estimated cost relative to the optimized LOTUS join program. UQE, which uses an embedding-based optimization, obtains similar accuracy to the search baseline, remaining well below the RP@5 and RP@10 of the LOTUS programs despite a generous LLM call budget, indicating that the UQE method has lower sample efficiency on this task. Next, we turn to a detailed comparison of DocETL and LOTUS, shown in Table 3 and Table 4. LOTUS consistently matches or exceeds the accuracy of DocETL's successful runs. Notably, the DocETL agentic optimizer requires GPT-4o and frequently fails, requiring multiple reruns, while LOTUS' optimizer provides statistical accuracy guarantees, which we provide a detailed analysis on in Section 5.5. Table 4 also shows that the LOTUS and DocETL programs use a similar number of total LLM calls and execution time, however the token consumption per call of the DocETL programs are lower on average due to its optimized plan, which leads to modestly lower execution time despite modestly higher LLM calls.

Lastly, we turn to Table 5 and study the candidate plans produced by the LOTUS join program. Plan 1, the sim-filter, requires more LLM calls to meet the recall and precision targets, compared to Plan 2, which is the project-sim-filter plan. This reflects that Plan 2's proxy offers a stronger signal for predicate evaluations. We see that the LOTUS optimizer selects Plan 2 to execute since it is lower cost, requiring fewer LLM calls. The selected pattern also results in significantly better accuracy due to its higher quality proxy signal for this task. Specifically, the project-sim-filter pattern offers 37% higher RP@5 and 25% higher RP@10 compared to the sim-filter, with $1,000\times$ fewer LLM calls than the gold algorithm.

5.3 Search & Ranking

Relevance-based ranking has been widely studied in the context of information retrieval. In addition, our conversations with LOTUS users reveal a common need for ranking based on *complex natural language criteria* (e.g., ranking customer reviews based on how frustrated they sound). In this application, we assess LOTUS' ranking capabilities on both types of ranking tasks with two datasets: BEIR's SciFact test set [63], a widely used benchmark for retrieval and re-ranking, and a new dataset, HellaSwag-bench, which we generated to assess more complex ranking tasks. For both we report average nDCG@10, a standard ranking metric, and execution time (ET). The SciFact dataset consists of scientific claims and a corpus of articles, where the task is to rank articles by relevance to a given claim. We sample 300 claims for our evaluation. HellaSwag-bench consists of 200 synthetic paper abstracts[8], each reporting an accuracy on the HellaSwag dataset [74], and the task is to rank abstracts according to their reported accuracy. This dataset provides an objective ground truth, while focusing on a reasoning-based ranking criteria, rather than a relevance-based one[9]. We report results for $n = 20$ trials at temperature $t = 0.7$, similar to prior works [41].

Table 6: Ranking Results on SciFact and Hellaswag-bench Datasets

Method	SciFact			HellaSwag-bench		
	nDCG@10	ET (s)	# LM Calls	nDCG@10	ET (s)	# LM Calls
Search	0.712	0.009	0	0.119	0.008	0
Reranker	0.741	2.64	0	0.461	2.36	0
AI UDF	0.457	9.83	100	0.091	19.7	200
DocETL - Llama 70B optimizer (avg. of 3 sucesses)*	N/A	N/A	N/A	0.246	14.1	3
DocETL - GPT 4o optimizer*	N/A	N/A	N/A	N/A	N/A	N/A
LOTUS	0.765	36.3	213.2	0.919	57.0	506.4

* DocETL is unable to acheive any successful runs using GPT-4o for its optimizer on SciFact and HellaSwag-bench, nor using Llama-70B for its optimizer on SciFact

Table 7: Comparison of LOTUS' Top-k to the Gold Algorithm and Other High-Quality Top-k Algorithms for Ranking

Method	SciFact			HellaSwag-bench		
	nDCG@10	ET (s)	# LM Calls	nDCG@10	ET (s)	# LM Calls
Quadratic Topk	0.768	801.2	4950	0.886	2116.6	19,900
Heap Topk	0.765	60.0	192.6	0.896	59.4	241.5
Quickselect Topk	0.771	40.5	237.0	0.893	49.4	448.1
LOTUS	0.765	36.3	213.2	0.919	57.0	506.4

Baselines. In addition to the standard search baseline, we add a re-ranking method using the MixedBread cross-encoder [15], a high quality re-ranker that is common in relevance-based retrieval. The AI UDF baseline performs a pointwise-ranking by prompting the LLM to assign a relevance score to each row, similar to prior work [76]. The DocETL baseline uses the system's LLM `reduce` operation for ranking, as described by the pre-print [61]. Our LOTUS program uses the `sem_topk` operator with a semantic index on the document corpus of both datasets. For the reranker, AI UDF, DocETL, and LOTUS programs on SciFact, we perform search to retrieve 100 articles, before re-ranking them with each method.

Results. We evaluated the performance of the LOTUS program against baselines, in comparison to the gold algorithm, and to alternative high-quality ranking algorithms we considered as gold algorithms for our `sem_topk`.

First Table 6 demonstrates that LOTUS achieves $3\%-100\%$ higher accuracy than the next best-performing baselines on Scifact and HellaSwag-bench. Specifically, as expected, the re-ranker offers competitive accuracy compared to the LOTUS program on Scifact, which focuses on relevance-based retrieval, the supervised task the re-ranker model was trained on. However, on HellaSwag-bench,

[8]Each paper abstract in HellaSwag-bench is synthetically generated by prompting Llama-70b to write a research abstract that claims a specified accuracy value, which we randomly sample from $0-100\%$.

[9]While the Hellaswag-bench ranking task might be efficiently solved with a sem_map, to extract accuracy values, followed by a structured sorting, our evaluation focuses on assessing semantic ranking algorithms. We find this benchmark useful for understanding performance trade-offs, which may generalize to a wider set of reasoning-based ranking queries.

(μ_1)	Advancements in Recommender Systems and Multimodal Data Integration
(μ_2)	Advancements in Generative Information Retrieval Systems
(μ_3)	Advancements in Large Language Models for Various Applications
(μ_4)	Advancements in AI Security and Malware Detection Techniques
(μ_5)	Advancements in Robotic Navigation and Manipulation Techniques

Figure 7: Discovered group labels from LOTUS *sem_group_by("the topic of each {paper}", C=5)* over a dataset of recent ArXiv papers, scraped from the database (cs.DB), information retrieval (cs.IR), crytography and security (cs.CR) and robotics (cs.RO) domains.

the LOTUS program significantly outperforms the re-ranker on this reasoning-based task. We also observe that the point-wise AI UDF method and the list-wise ranking, used by DocETL, offer substantially lower accuracy, unable to outperform the re-ranking baseline. This reflects the limitations of point-wise and list-wise ranking, which has been studied in prior work [58]. Notably, the DocETL baseline altogether fails to produce any successful execution plans using the GPT-4o optimizer on Scifact and Hellaswag-bench and using the Llama-70B optimizer on Scifact.

Turning our attention to Table 7, we compare the quick-select top-k gold algorithm to several alternative candidates. The quadratic top-k, heap top-k, and quick-select top-k offer comparable accuracy, making each a viable gold algorithm. However, the three candidate choices provide significant differences in efficiency. The quadratic algorithm, which performs an LLM comparison between each pair of input documents, requires $20 - 82\times$ more LM calls and over $10\times$ higher execution time than the heap top-k and quick-select top-k. In addition, the heap top-k and quick-select top-k methods offer interesting trade-offs in LLM call complexity and execution time. Notably the quick-select top-k offers $16-32\%$ lower execution time than the heap-based sorting method, despite requiring more LLM calls in some cases. This is because the quick-select top-k implementation allows for efficient batched processing in each round of the algorithm, whereas the heap-based top-k incurs sequential LLM calls during heap updates. LOTUS currently implements the quick-select topk-k by default, favoring lower execution time, although we future iterations may leverage multiple algorithms.

Finally, the Table 7 compares the quick-select top-k with the LOTUS quick-select top-k using embedding-based pivot selection. We demonstrate that the optimization is lossless, and can decrease latency by $10\%s$. Specifically, this optimization is useful where document rank correlates with semantic similarity, which is likely on the Scifact dataset.

5.4 Topic Analysis on ArXiv Papers

A common unsupervised discovery task requires grouping a large document corpus by topics and assigning descriptive labels to each group. We consider this task over a dataset of 647 recent ArXiv articles, which we scraped from the database (cs.DB), information retrieval (cs.IR), cryptography and security (cs.CR), and robotics (cs.RO) domains. We aim to discover 5 key topic labels.

We can succinctly represent this task using the `sem_group_by` operator with LOTUS. AI UDFs, DocETL and standalone search systems do not serve operations for unsupervised semantic-based group discovery. While UQE [25] studies a tractable optimization for serving non-LLM aggregations, e.g., COUNT, with LLM-based group-bys using stratified sampling, it does not focus on implementing, optimizing, and evaluating a standalone group-by operator,

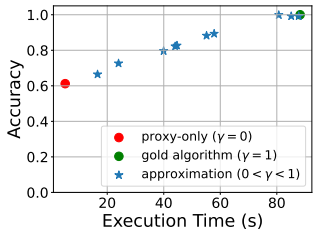


Figure 8: Classification accuracy vs. execution time (s) for the LOTUS sem_group_by over the ArXiv dataset. We compare the performance of implementing classifying each paper using the proxy only (red circle) or oracle only (green circle) to our approximation with varied accuracy targets, at $\delta = 0.2$ (stars).

but suggests a standalone group-by implementation similar to our `sem_group_by` gold algorithm. Thus, in this section, we focus our analysis on understanding the performance of our `sem_group_by` gold algorithm and analyzing our optimization with respect to it. We use Llama-70B and E5 embeddings. For approximations, we use a sample size of 100 and $\delta = 0.2$.

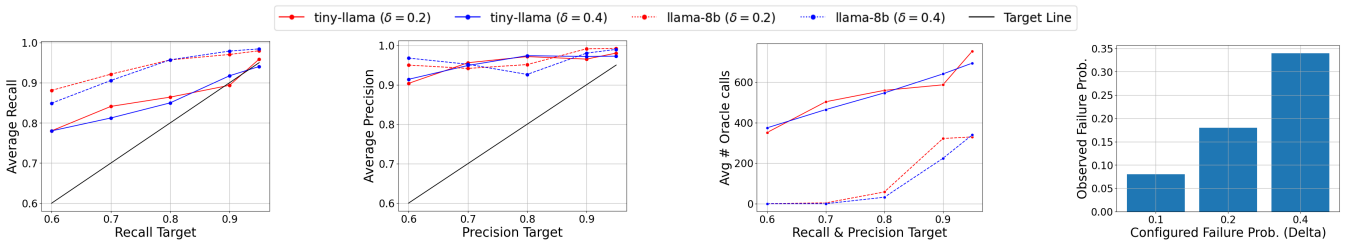
Results. The `sem_group_by` consists of two sub-tasks: (a) discovering representative group labels, and (b) classifying each document to a discovered label.

We first qualitatively analyze the labels discovered, in the first sub-task of the LOTUS implementation, which is equivalent to gold algorithm. As Figure 7 shows, the discovered labels intuitively align with recent research topics from the cs.DB, cs.IR, cs.CR, and cs.RO ArXiv domains related to recommendation systems, retrieval, LLM applications, AI security, and robotics. Performing this unsupervised group discovery took 44.03 seconds, representing a tractable implementation.

Turning to the classification sub-task, we compare the performance of the gold algorithm to our approximation, which uses an embedding-based proxy here. Figure 8 compares the execution time and classification accuracy of the oracle-based gold algorithm (green circle), our approximations using both the Llama-70B oracle LLM and embedding-based proxy at varied accuracy targets (blue stars), and the embedding-based proxy alone (red circle). We see that the proxy-only baseline is $17.4\times$ faster than the oracle, but about 39% less accurate. Our approximation algorithm allows users to declaratively interpolate between these two extremes by varying the accuracy target. We also note that the sampling-based optimization procedure used to perform this optimization took less than 5 seconds, representing a small relative cost.

5.5 Accuracy Guarantees Evaluation

Our approximation methods are designed to adaptively learn when to leverage a proxy to provide an accuracy guarantee, despite variations in proxy accuracy. To demonstrate the accuracy guarantees provided by our semantic operator optimizations, we study the `sem_filter` performance on the fact-checking performance with varied proxy models, recall and precision targets and failure probability parameters. Figure 9a and Figure 9b demonstrate the average accuracy observed over 20 trials for varied recall targets and precision targets. We evaluate the filter approximation with both the Llama-8B proxy (solid lines) and a weaker proxy, with TinyLlama-1B (dashed lines) with a failure probability of $\delta = 0.2$ (red) and



(a) Observed vs Target Recall

(b) Observed vs Target Precision

(c) # Oracle Calls vs Accuracy Target

(d) Observed vs Configured Failure Probability

Figure 9: We evaluate statistical accuracy guarantees with our semantic filter for fact-checking on the FEVER dataset. We show (a) average observed recall vs. configured recall targets (γ_R), (b) average observed precision vs. configured precision targets (γ_P), and (c) the number of oracle LLM calls vs. configured accuracy targets ($\gamma_R = \gamma_P$), using a Llama-70B oracle and either a Llama-8B proxy (dashed lines) or a weaker TinyLlama proxy (solid lines), for two different failure probabilities ($\delta = 0.2, 0.4$). We also show observed vs. configured failure probabilities using the TinyLlama proxy and Llama-70B oracle, for accuracy targets $\gamma_R = \gamma_P = 0.9$.

$\delta = 0.4$ (blue). As expected, the average recall and precision values increase with increasing accuracy targets for either metric. Figure 9c demonstrates that decreasing the recall and precision target tends reduce the number of calls routed to the oracle. As expected, at a fixed accuracy target, our approximation requires more oracle calls when using the weaker proxy, in this case TinyLlama. Finally, in Figure 9d we record the percentage of 50 trials that do not meet the recall and precision target, both set to 0.9, using the TinyLlama proxy. The results confirm that the observed failure probabilities increase with the user-configured delta values, and are lower than the configured failure probabilities, as expected due to our conservative implementation.

6 RELATED WORK

Data Systems for Row-wise LLM Operations. Many prior works focus on map-like, *row-wise LLM-based operators*, either with generic AI user-defined functions (UDFs) or specialized operators [18, 46, 47, 50]. AI UDF systems [1, 8, 14, 49, 53, 67] provide a low-level, non-declarative programming interface for row-wise LLM invocations. These systems also integrate vector search, equivalent to our `sem_search` or `sem_sim_join`, and support SQL queries that combine row-wise AI UDFs with non-LLM operations (e.g. average, count). Alternatively, several recent works implement optimized row-wise LLM operations for *specialized* tasks, including data cleaning, extract-transform-load (ETL) and conversational agents. ZenDB [46], EVAPORATE [18] and Palimpzest [47] provide LLM-based functions for extracting semi-structured documents into structured tables, logically similar to our `sem_extract`, and Palimpzest also implements an LLM-powered `filter`, logically similar to our `sem_filter`. SUQL [50] augments SQL with a two logically row-wise operators, `answer`, for questions-answering over each row, and `summary`, for *row-wise* LLM-based summarization, to serve knowledge-grounded *conversational agents*. In contrast to these works, semantic operators are a *general-purpose query model* for semantic transformations over datasets, including both logically row-wise ones and more complex ones (e.g., joins, aggregations and ranking). Our detailed evaluation (Section 5) demonstrates the performance limitations of row-wise LLM execution models.

Best-Effort LLM-Based Analytics Systems. Several recent systems [17, 25, 61] go beyond row-wise LLM operators. However, these systems lack a formalism for LLM-based operators, providing no performance guarantees for their execution. Following a preprint [54] of this work, DocETL [61] proposes to use LLM agents as the query optimizer over LLM-powered operators. This is a promising direction for future work; however, our evaluation (Section 5) demonstrates that DocETL's agentic approach leads to high variance in performance, no accuracy guarantees, and requires multiple reruns to achieve comparable accuracy to ours, with approximately 1 in 3 runs failing due to optimizer errors, indicating that more work is needed to make these methods robust. UQE [25] also studies optimizations for LLM-powered operators, proposing an embedding-based approximation for LLM-powered filters. In contrast to our methods, which adaptively learn how to leverage a given proxy to meet accuracy targets, UQE's approach provides best-effort performance with no accuracy guarantees. UQE also studies optimizations for non-LLM aggregations (e.g., average, count) combined with LLM-based filter and group-by operators using stratified sampling, which is complementary to this work.

General ML-based Query Processing. Prior works study the use of non-LLM machine learning (ML) in databases. In contrast, our work integrates LLMs, which motivates our new formalisms for language-based operators and novel optimizations. MADLib [32] extends SQL with abstractions for descriptive statistics and machine learning (e.g., regression and classification). NoScope [36], TASTI [40], SUPG [37], BlazeIt [35] and probabilistic predicates [51] propose methods to optimize queries involving expensive ML-based predicates (e.g., using object detectors) over large datasets, typically in video analytics. Some optimizations proposed in these works, such as model cascades and predicate re-ordering, are also useful for optimizing LOTUS pipelines with language models.

Table Question Answering. A large body of work, including retrieval-augmented generation [44] and Text2SQL [70, 71, 73, 75], serves natural language interfaces for question-answering over databases. Typically, the LLM system invokes external tools, e.g., search APIs or SQL programs. Interestingly, these agentic workflows can instead invoke semantic operator programs as the underlying API for data processing. Recent work [19] demonstrates the promise of this approach to outperform baseline TableQA methods.